Implementation of a CI/CD Pipeline Using Jenkins and Terraform

Name – Atharva Gavali
PRN – 202301070040

1. Objective

To implement a basic CI/CD workflow using Jenkins and provision cloud infrastructure using Terraform. Jenkins integrates source code from GitHub, builds the Python application, runs automated tests and deploys it. Terraform provisions an AWS S3 bucket with versioning, encryption and public-access restrictions.

2. Tools and Technologies

Component	Technology
Source control	Git, GitHub
CI/CD	Jenkins
Application	Python 3.11, Flask, pytest
Infrastructure as Code	Terraform 1.14.9
Cloud	Amazon Web Services – S3
Environment	Windows 11, PowerShell, VS Code

3. Implementation

3.1 Application Development

A Flask application was created with a home route (/) and a health-check route (/health). The health endpoint returns a JSON response showing the application status. Automated tests were implemented using pytest.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return jsonify({
        "message": "Welcome to the Jenkins CI/CD Demo!",
        "status": "Application Running",
        "technology": "Python Flask"
    })

@app.route("/health")
def health():
    return jsonify({"status": "healthy"})
```

Local and pipeline test execution reported 2 tests passed.

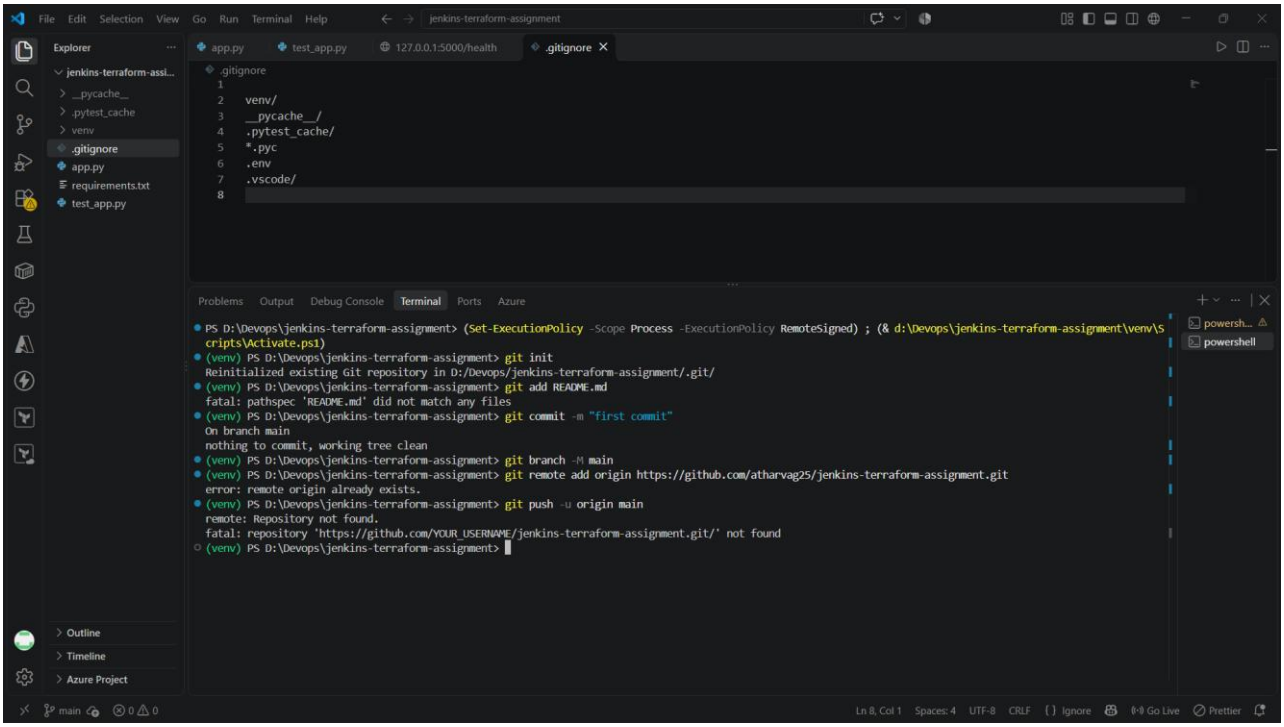


Figure 1. Project files and local application health endpoint

3.2 Jenkins Pipeline Configuration

A Jenkins Pipeline job named Flask-CI-CD-Pipeline was connected to the public GitHub repository. The Jenkinsfile defines four stages:

- Source – checks out the repository from GitHub.
- Build – creates a Python virtual environment and installs requirements.txt.
- Test – executes pytest in the virtual environment.
- Deploy – runs deploy.ps1 and verifies the Flask /health endpoint.

The deployment entry point starts Flask on 127.0.0.1:5001. The PowerShell script checks whether the application is already healthy; otherwise, it starts the process and polls the health endpoint.

```
from app import app

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5001)
```

After the successful Jenkins deployment, the Flask home route was verified in a browser at http://127.0.0.1:5001. The page returned the expected JSON response confirming that the application was running.

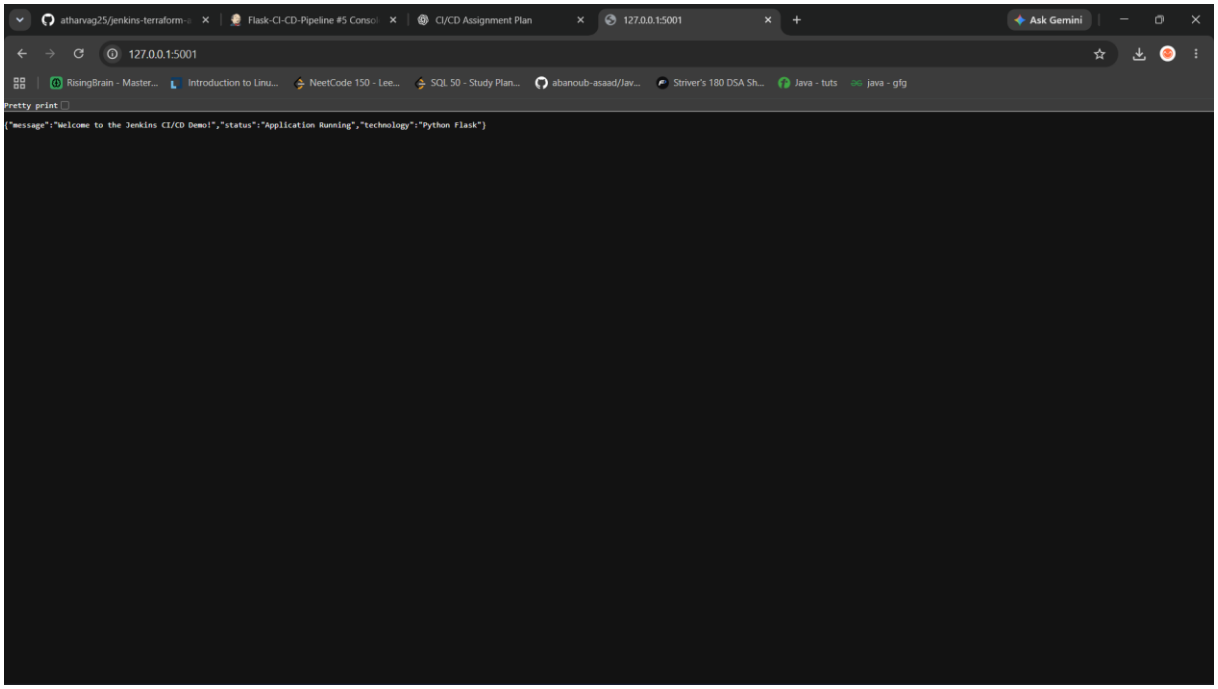


Figure 5. Successfully deployed Flask application displaying the expected home-route JSON response.

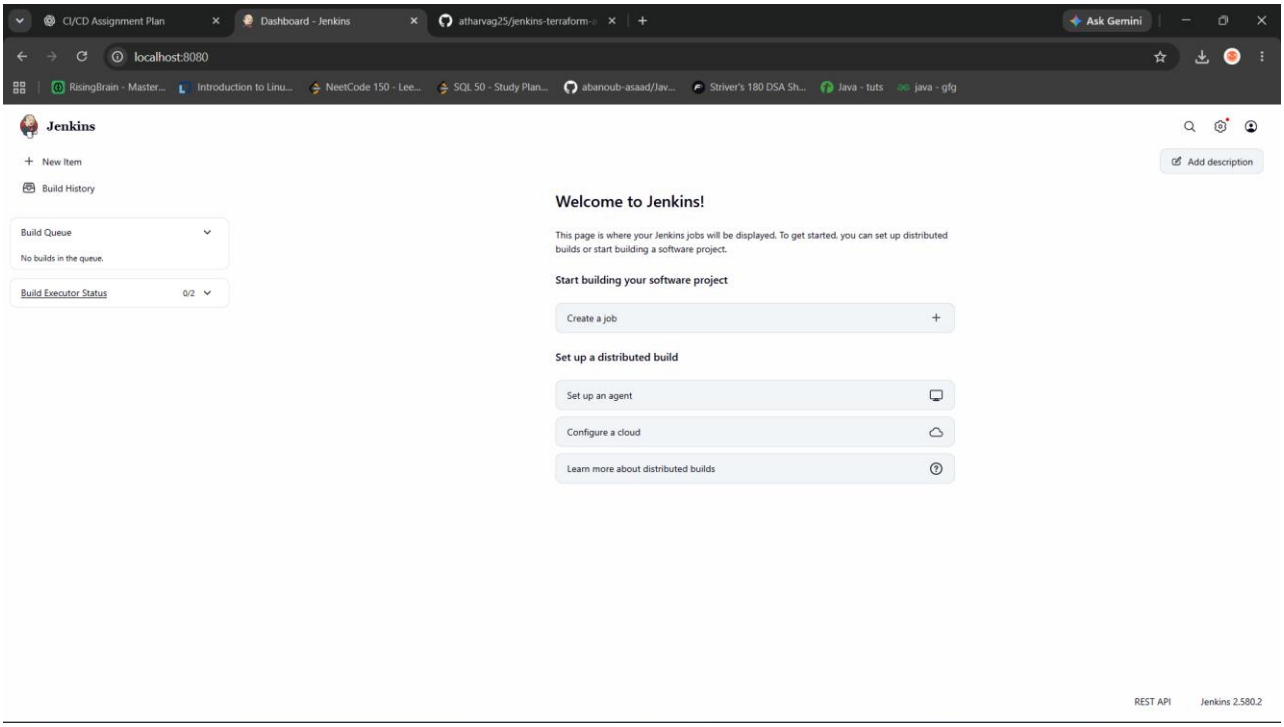


Figure 2. Jenkins interface before pipeline execution

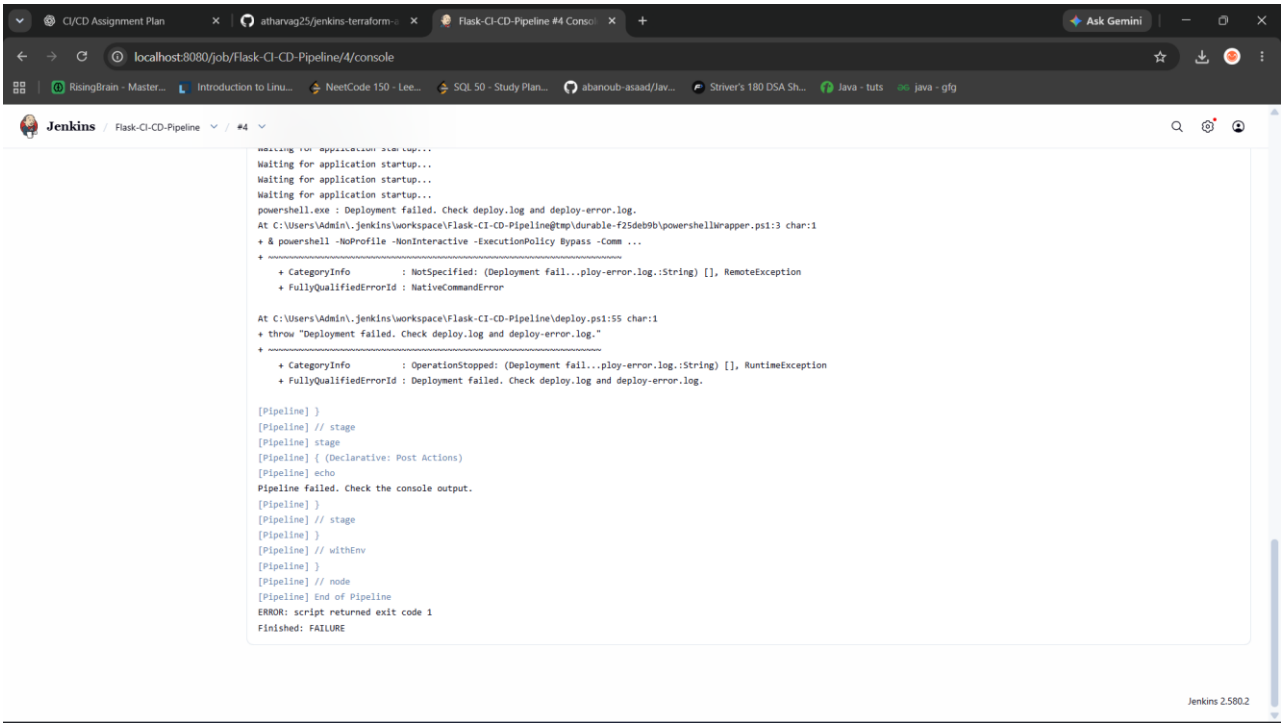


Figure 3. Jenkins pipeline execution and console output

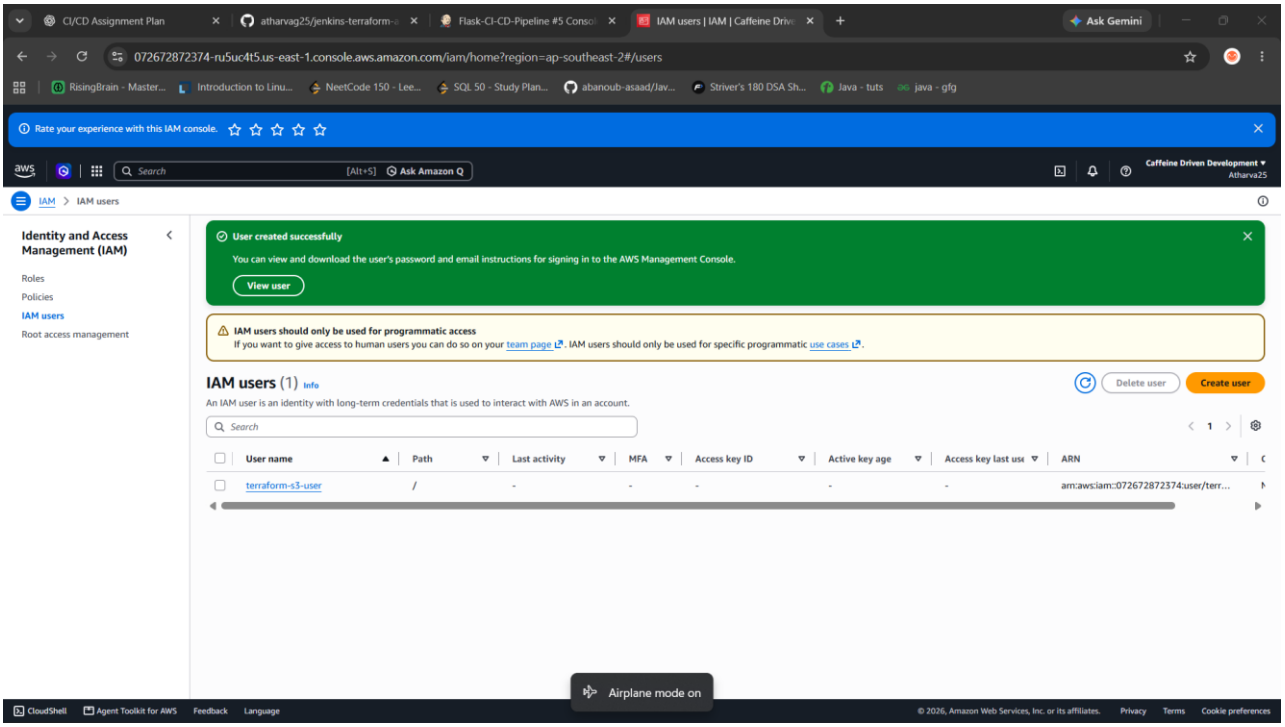


Figure 4. Jenkins build result / pipeline verification

3.3 Terraform Configuration

Terraform was configured to create an S3 bucket in the assigned AWS region, ap-southeast-2. The configuration enables bucket versioning, AES-256 server-side encryption and all four S3 public-access-block controls.

```
terraform {
 required_version = ">= 1.5.0"
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 6.0"
  }
 }
```

```
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "project_bucket" {
  bucket = var.bucket_name
  tags = {
    Name        = var.bucket_name
    Environment = "Development"
    Project     = "Jenkins-Terraform-CICD"
    ManagedBy   = "Terraform"
  }
}

resource "aws_s3_bucket_versioning" "project_versioning" {
  bucket = aws_s3_bucket.project_bucket.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "project_encryption" {
  bucket = aws_s3_bucket.project_bucket.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "project_public_access" {
  bucket = aws_s3_bucket.project_bucket.id
  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}

variable "aws_region" {
  description = "AWS region for S3 bucket provisioning"
  type        = string
  default     = "ap-southeast-2"
}

variable "bucket_name" {
  description = "Globally unique S3 bucket name"
  type        = string
  default     = "atharvag25-terraform-cicd-demo-2026"
}

output "s3_bucket_name" {
  value = aws_s3_bucket.project_bucket.bucket
}
output "s3_bucket_arn" {
  value = aws_s3_bucket.project_bucket.arn
}
output "s3_bucket_region" {
  value = var.aws_region
}
```

Terraform init installed the AWS provider. terraform fmt and terraform validate completed successfully. The final apply completed with 4 resources added, 0 changed and 1 previous bucket instance destroyed/recreated.

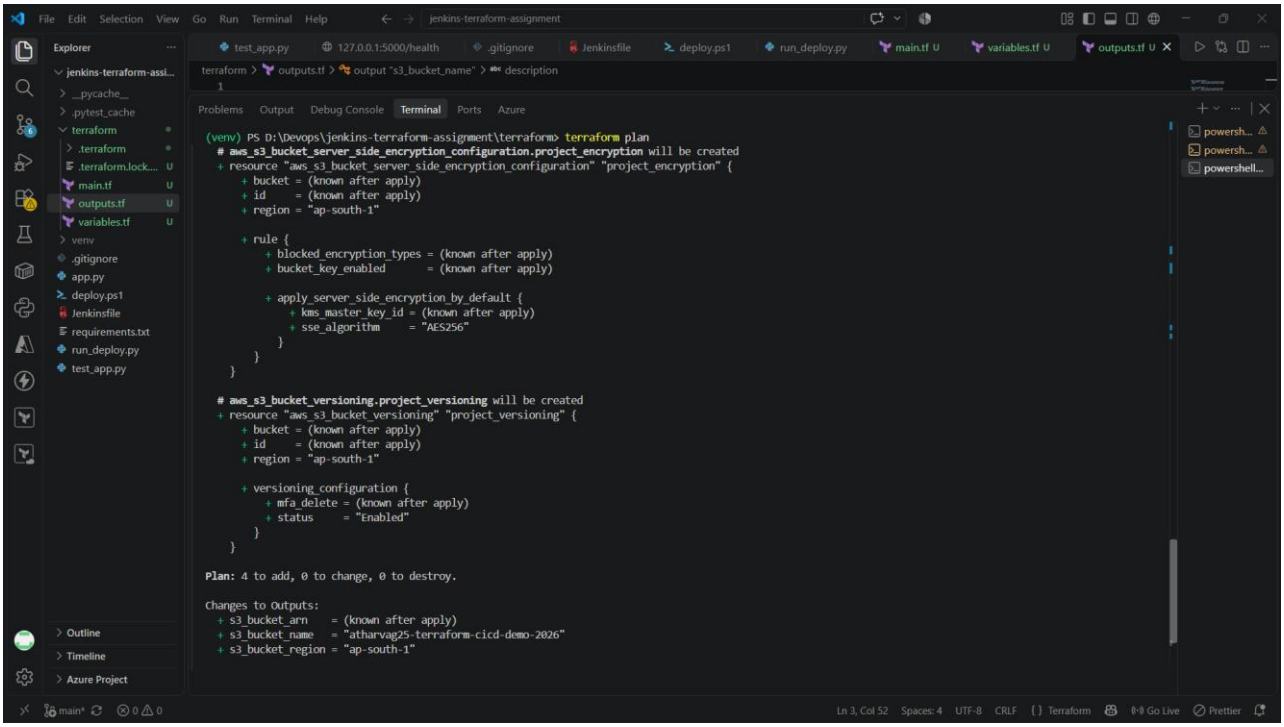


Figure 6. Terraform plan showing the S3 resources

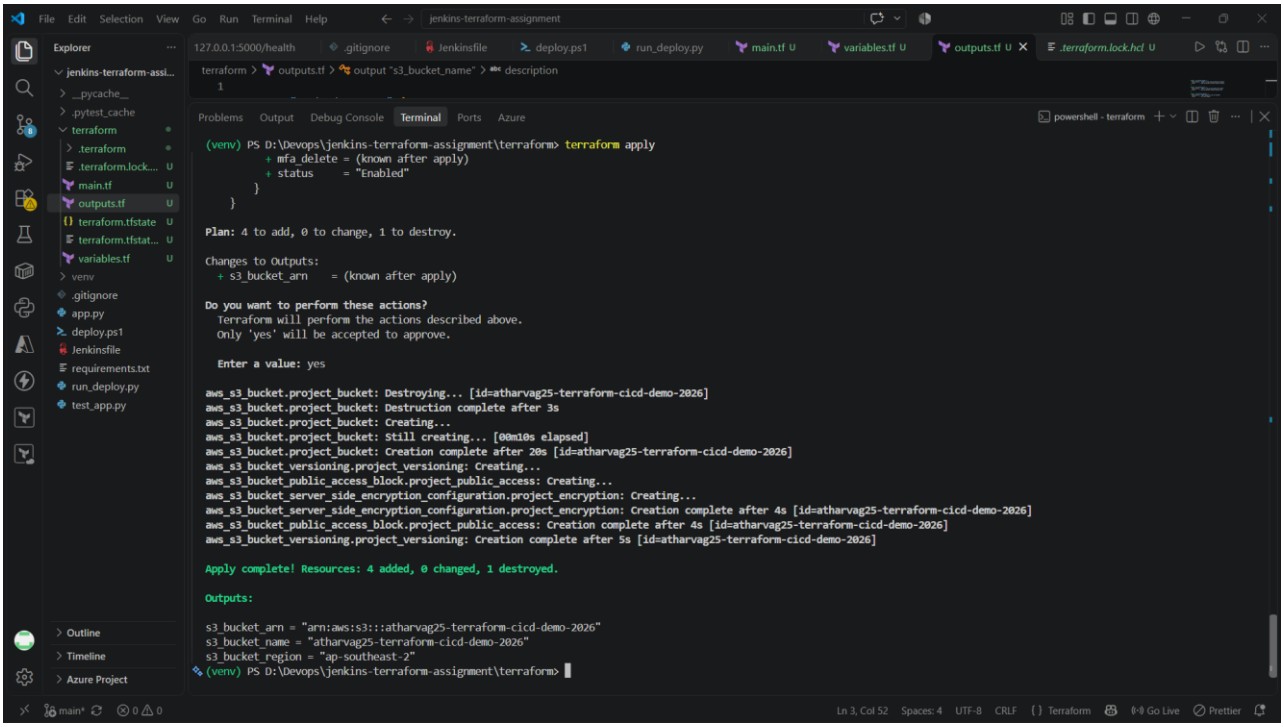


Figure 7. Successful Terraform apply and output values

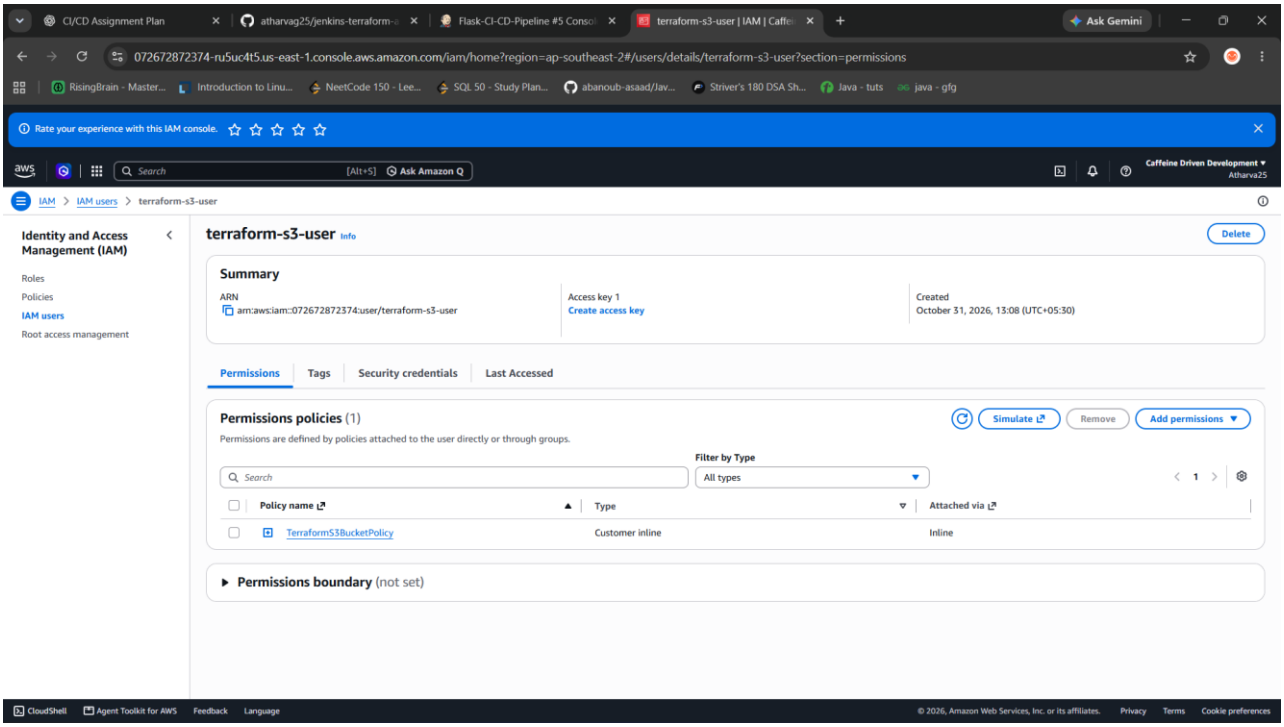


Figure 8. AWS IAM user and attached permissions used for provisioning

4. Verification and Results

The Jenkins pipeline completed successfully. The test stage reported 2 passed tests, and the deployment stage verified the Flask health endpoint. Terraform successfully provisioned the S3 bucket and its associated configuration resources.

Check	Result
Jenkins pipeline	SUCCESS
Automated tests	2 passed
Application health	healthy
Terraform validation	Successful
Terraform apply	4 added, 0 changed, 1 destroyed/recreated
AWS region	ap-southeast-2

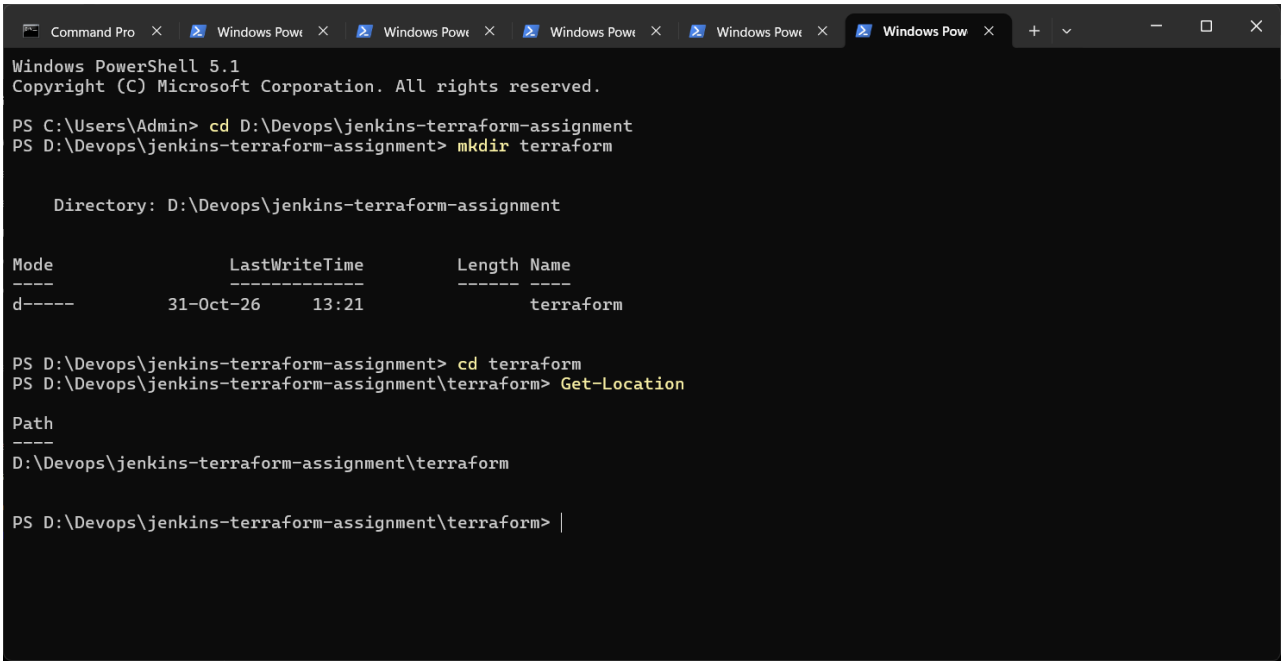


Figure 9. Terraform working directory and configuration setup

5. Source Code Repository

The application, Jenkins pipeline, deployment scripts and Terraform configuration are maintained in the following public GitHub repository:

https://github.com/atharvag25/jenkins-terraform-assignment

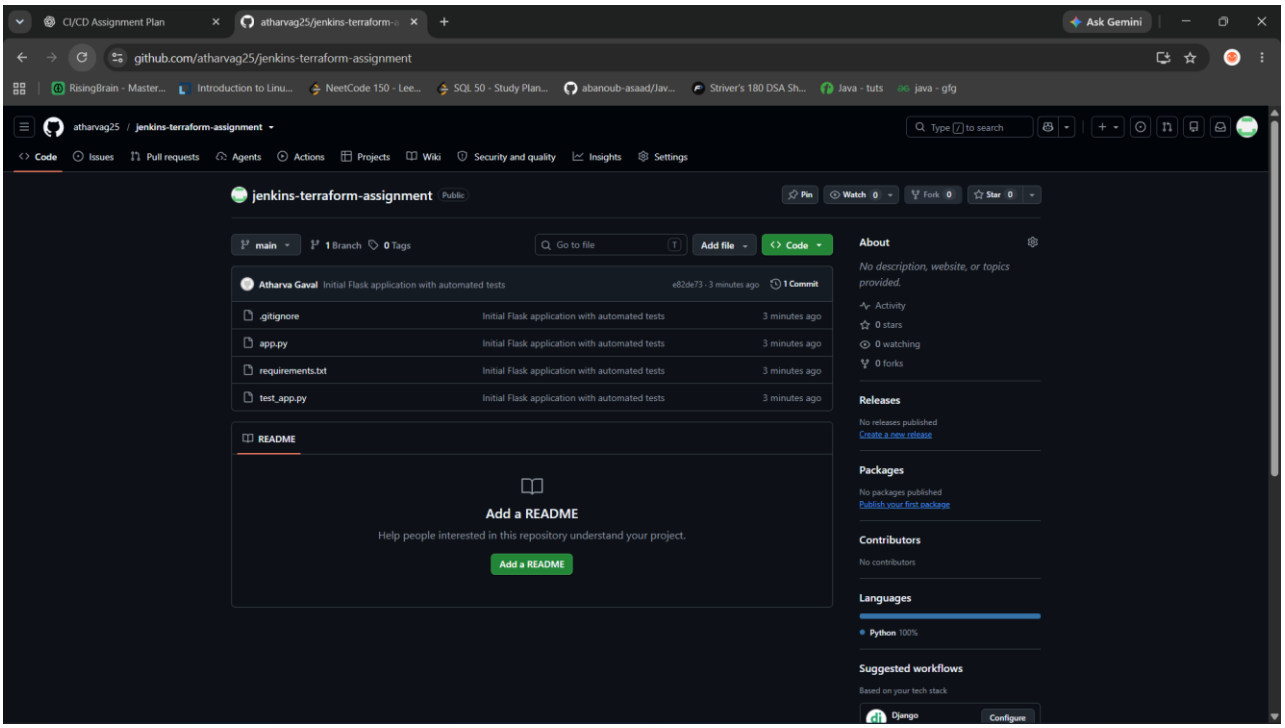


Figure 10. GitHub repository containing the assignment implementation

6. Conclusion

The assignment demonstrates source integration, automated build and testing, deployment verification through Jenkins, and AWS infrastructure provisioning through Terraform. The CI/CD pipeline and S3 infrastructure configuration were successfully implemented.